Process & Decision Documentation

Project/Assignment Decisions

Side Quests and A4 (Individual Work)

One significant decision I made was to change the idea of the “H to Hide” to “H to Show”, with AI’s generated output, it didn’t make sense why the debug screen would get darker when you click H. Only when you press “H” will you be able to toggle for further action, this made no sense from a user perspective so I changed it so that it would align better with user expectations.

When clicking H to hide, the blac rectangle just becomes slightly more transparent, but I feel this not give the effect of what it should be like. If it says H to hide, it should be more hidden.

GenAI Documentation

If GenAI was used (keep each response as brief as possible):

Date Used: March 22 - 23, 2026

Tool Disclosure: Gemini

Purpose of Use: Used it to brainstorm during the ideation process to create the features and to create the debug screen.

Summary of Interaction: AI contributed to the base code for the debug screen and the toggling portion of the menu.

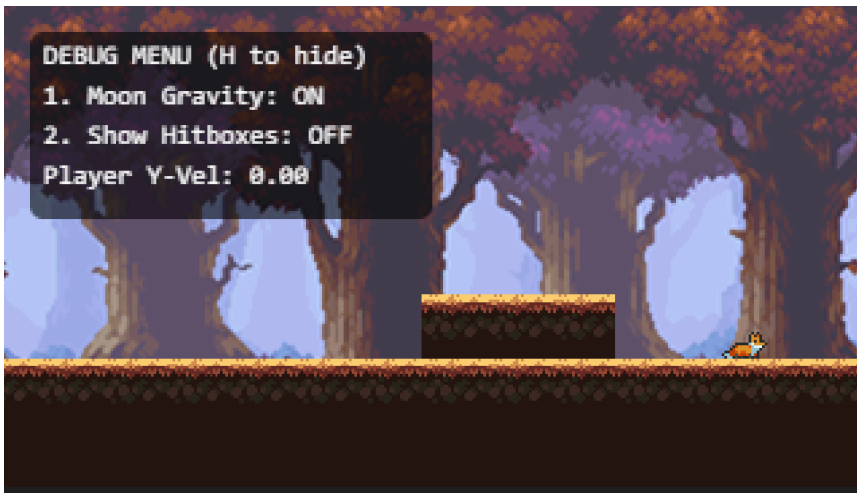
Human Decision Point(s): I changed AI’s output in terms of “H to Hide” to “H to Show” to reflect user expectations, I also changed the alpha values of the fills for the debug menu to show a greater difference when clicking H to show the debug menu.

Integrity & Verification Note: I checked GenAI output based on sidequest instructions and reviewing the slideshow to see if it looked like what was expected for this week.

Scope of GenAI Use: AI did not contribute to the change in debug menu for the alpha fills and it also did not contribute to change in features for the “H to Show”.

Limitations or Misfires: AI did not work well to support user expectations, the output made no sense as user because of the misleading title “H to Hide” and also the debug menu transparency change (it got darker when it should’ve been hidden).

The first image is of the original hidden state, and the second image reflects the new state after my changes.



Summary of Process (Human + Tool)

I did a lot of testing and revising the game in the live server to determine the appropriate value fills for the alpha to ensure the transparency made sense. I also revised the slideshow and the examples on there to determine whether the debug menu looked right and was what was expected.

Decision Points & Trade-offs

One key decision was changing the text from “H to Hide” to “H to Show” as it made more sense for user expectations. I thought to keep it “H to Hide” at first and tried to make the initial start darker so that when you click H, it would get more transparent. This process confused me a little because of the way the debug screen would layer on top of each other, when I would change the alpha values to make the initial one darker, and after you click H to be lighter, it would just

layer on top of the initial value, which made little to no difference. I decided to change the title to “H to Show” instead and make it so after you click H it would be darker rather than lighter.

Verification & Judgement

This decision felt appropriate after revising it when play testing it using the live server as it better reflected user expectations. After the change, the initial view is of the debug menu being in a “hidden” state and after you click “H”, the menu would become more visible and you would be able to toggle the moon gravity and the visibility of Hit boxes. This made more sense than the original which made me feel that the decision was more appropriate and suitable to prevent user confusion.

Limitations, Dead Ends, or Open Questions

The only thing that didn’t work as expected was the way the debug menu appeared, I didn’t necessarily resolve the issue of it getting layered when I imputed it again, I couldn’t really figure out a way to remove the first call of the menu and replace it with the second call so I left it and edited the code to make it work.

Appendix

What I Gave AI:

Based on this slideshow and the code below, where do I the new code and what do I edit based on this task -> add a debug screen to Week 9 Example 3 that allows you to toggle moon gravity on/off, and at least one other potential feature you might need in developing a game. These are the codes, Index.html <!doctype html>

```
<html lang="en">

  <head>

    <meta charset="utf-8" />

    <title>GBDA302 Week 9 - Example 3: Adding Sound & Music</title>

    <style>

      body {

        margin: 0;

        overflow: hidden;

        background: #222;
```

```
      }
      canvas {
        display: block;
      }
    </style>
  </head>
  <body>
    <script src="https://cdn.jsdelivr.net/npm/p5@1.11.4/lib/p5.min.js"></script>
    <script src="https://cdn.jsdelivr.net/npm/p5@1.11.4/lib/addons/p5.sound.min.js"></script>
    <script src="https://p5play.org/v3/planck.min.js"></script>
    <script src="https://p5play.org/v3/p5play.min.js"></script>
    <script src="sketch.js"></script>
  </body>
</html>
jsconfig.json {
  "compilerOptions": {
    "target": "es6",
    "checkJs": false,
    "allowJs": true,
  },
  "include": [
    "*.js",
    "**/*.js",
    "./types/p5/index.d.ts",
```

```
    "./types/p5/global.d.ts"
  ],
   "exclude": ["node_modules"]
}
```

sketch.js /*

```
  Week 9 — Example 3: Adding Sound & Music


  Course: GBDA302 | Instructors: Dr. Karen Cochrane & David Han
  Date: Mar. 19, 2026


  Controls:
    A or D (Left / Right Arrow)   Horizontal movement
    W (Up Arrow)                  Jump
    Space Bar                     Attack


  Tile key:
    g = groundTile.png       (surface ground)
    d = groundTileDeep.png   (deep ground, below surface)
      = empty (no sprite)
*/


let player;
let playerImg, bgImg;
let jumpSfx, musicSfx;
```

```
let musicStarted = false;


let playerAnis = {
  idle: { row: 0, frames: 4, frameDelay: 10 },
  run: { row: 1, frames: 4, frameDelay: 3 },
  jump: { row: 2, frames: 3, frameDelay: Infinity, frame: 0 },
  attack: { row: 3, frames: 6, frameDelay: 2 },
};


let ground, groundDeep;
let groundImg, groundDeepImg;


let attacking = false; // track if the player is attacking
let attackFrameCounter = 0; // tracking attack animation


// --- TILE MAP ---
// an array that uses the tile key to create the level
let level = [
  "              ",
  "              ",
  "              ",
  "              ",
  "              ",
  "       ggg    ",
```

```
  "gggggggggggggg", // surface ground
  "dddddddddddddd", // deep ground
];

// --- LEVEL CONSTANTS ---
// camera view size
const VIEWW = 320,
  VIEWH = 180;

// tile width & height
const TILE_W = 24,
  TILE_H = 24;

// size of individual animation frames
const FRAME_W = 32,
  FRAME_H = 32;

// Y-coordinate of player start (4 tiles above the bottom)
const MAP_START_Y = VIEWH - TILE_H * 4;

// gravity
const GRAVITY = 10;

function preload() {
```

```
  // --- IMAGES ---
  playerImg = loadImage("assets/foxSpriteSheet.png");
  bgImg = loadImage("assets/combinedBackground.png");
  groundImg = loadImage("assets/groundTile.png");
  groundDeepImg = loadImage("assets/groundTileDeep.png");

  // --- SOUND ---
  if (typeof loadSound === "function") {
    jumpSfx = loadSound("assets/sfx/jump.wav");
    musicSfx = loadSound("assets/sfx/music.wav");
  }
}

function setup() {
  // pixelated rendering with autoscaling
  new Canvas(VIEWW, VIEWH, "pixelated");

  // needed to correct an visual artifacts from attempted antialiasing
  allSprites.pixelPerfect = true;

  world.gravity.y = GRAVITY;

  // Try to start background music immediately.
  if (musicSfx) musicSfx.setLoop(true);
```

```
startMusicIfNeeded();

// --- TILE GROUPS ---
ground = new Group();
ground.physics = "static";
ground.img = groundImg;
ground.tile = "g";

groundDeep = new Group();
groundDeep.physics = "static";
groundDeep.img = groundDeepImg;
groundDeep.tile = "d";

// a Tiles object creates a level based on the level map array (defined at the beginning)
new Tiles(level, 0, 0, TILE_W, TILE_H);

// --- PLAYER ---
player = new Sprite(FRAME_W, MAP_START_Y, FRAME_W, FRAME_H); // create the player
player.spriteSheet = playerImg; // use the sprite sheet
player.rotationLock = true; // turn off rotations (player shouldn't rotate)

// player animation parameters
player.anis.w = FRAME_W;
player.anis.h = FRAME_H;
```

```
  player.anis.offset.y = -4; // offset the collision box up
  player.addAnis(playerAnis); // add the player animations defined earlier
  player.ani = "idle"; // default to the idle animation
  player.w = 18; // set the width of the collsion box
  player.h = 20; // set the height of the collsion box
  player.friction = 0; // set the friciton to 0 so we don't stick to walls
  player.bounciness = 0; // set the bounciness to 0 so the player doesn't bounce

  // --- GROUND SENSOR --- for use when detecting if the player is standing on the ground
  sensor = new Sprite();
  sensor.x = player.x;
  sensor.y = player.y + player.h / 2;
  sensor.w = player.w;
  sensor.h = 2;
  sensor.mass = 0.01;
  sensor.removeColliders();
  sensor.visible = false;
  let sensorJoint = new GlueJoint(player, sensor);
  sensorJoint.visible = false;
}

function startMusicIfNeeded() {
  if (musicStarted || !musicSfx) return;
```

```
  const startLoop = () => {
    if (!musicSfx.isPlaying()) musicSfx.play();
    musicStarted = musicSfx.isPlaying();
  };

  // Some browsers require a user gesture before audio can start.
  const maybePromise = userStartAudio();
  if (maybePromise && typeof maybePromise.then === "function") {
    maybePromise.then(startLoop).catch(() => {});
  } else {
    startLoop();
  }
}

function keyPressed() {
  startMusicIfNeeded();
}

function mousePressed() {
  startMusicIfNeeded();
}

function touchStarted() {
  startMusicIfNeeded();
```

```
  return false;
}

function draw() {
  // --- BACKGROUND ---
  camera.off();
  imageMode(CORNER);
  image(bgImg, 0, 0, bgImg.width, bgImg.height);
  camera.on();

  // --- PLAYER CONTROLS ---
  // first check to see if the player is on the ground
  let grounded = sensor.overlapping(ground);

  // -- ATTACK INPUT --
  if (grounded && !attacking && kb.presses("space")) {
    attacking = true;
    attackFrameCounter = 0;
    player.vel.x = 0;
    player.ani.frame = 0;
    player.ani = "attack";
    player.ani.play(); // plays once to end
  }
```

```
// -- JUMP --
if (grounded && kb.presses("up")) {
  player.vel.y = -4;
  if (jumpSfx) jumpSfx.play();
}

// --- STATE MACHINE ---
if (attacking) {
  attackFrameCounter++;
  // Attack lasts ~6 frames * frameDelay 2 = 12 cycles (adjust if needed)
  if (attackFrameCounter > 12) {
    attacking = false;
    attackFrameCounter = 0;
  }
} else if (!grounded) {
  player.ani = "jump";
  player.ani.frame = player.vel.y < 0 ? 0 : 1;
} else {
  player.ani = kb.pressing("left") || kb.pressing("right") ? "run" : "idle";
}

// --- MOVEMENT ---
if (!attacking) {
  player.vel.x = 0;
```

```
    if (kb.pressing("left")) {
      player.vel.x = -1.5;
      player.mirror.x = true;
    } else if (kb.pressing("right")) {
      player.vel.x = 1.5;
      player.mirror.x = false;
    }
  }

  // --- KEEP IN VIEW ---
  player.pos.x = constrain(player.pos.x, FRAME_W / 2, VIEWW - FRAME_W / 2);
}
style.css html, body {
  margin: 0;
  padding: 0;
}

canvas {
  display: block;
}
```

What AI Gave Me:

To add a debug screen to Example 3, you should look at the design patterns shown in **Example 1: Debugging**, which uses a debug menu to inspect the game state and tune values in real time.

Based on the slideshow and your provided `sketch.js`, here is where to add the new code and what to edit:

1. Define Debug Variables

At the top of your `sketch.js` (near `let musicStarted = false;`), add variables to track the state of your debug features:

JavaScript

```
let showDebug = false; // Toggle the menu visibility

let moonGravity = false; // Toggle for moon gravity

let showHitboxes = false; // Additional feature: view collision boxes
```

2. Update Gravity Logic in `draw()`

In the `draw()` function, you need to edit how gravity is handled so it can change dynamically. Before the movement and jumping logic, add:

JavaScript

```
// Toggle gravity based on debug setting

if (moonGravity) {

  world.gravity.y = 2; // Low gravity (Moon)

} else {

  world.gravity.y = GRAVITY; // Normal gravity (Earth)

}



// Toggle hitbox visibility for all sprites

allSprites.debug = showHitboxes;
```

3. Add Input Toggles in `keyPressed()`

Modify the `keyPressed()` function to include keys that toggle the menu and the features. For example, use **'h'** to show the menu and numbers to toggle features:

JavaScript

```
function keyPressed() {
  startMusicIfNeeded();


  if (key === 'h' || key === 'H') {
    showDebug = !showDebug; // Toggle debug menu visibility
  }


  if (showDebug) {
    if (key === '1') moonGravity = !moonGravity; // Toggle Moon Gravity
    if (key === '2') showHitboxes = !showHitboxes; // Toggle Hitboxes
  }
}
```

4. Create the Debug UI in `draw()`

At the very end of your `draw()` function (after `player.pos.x` is constrained), add the code to render the visual menu:

JavaScript

```
 // --- DEBUG MENU UI ---
  if (showDebug) {
    camera.off(); // Ensure the menu stays fixed on screen
    fill(0, 0, 0, 150); // Semi-transparent background
    rect(10, 10, 150, 70, 5);
```

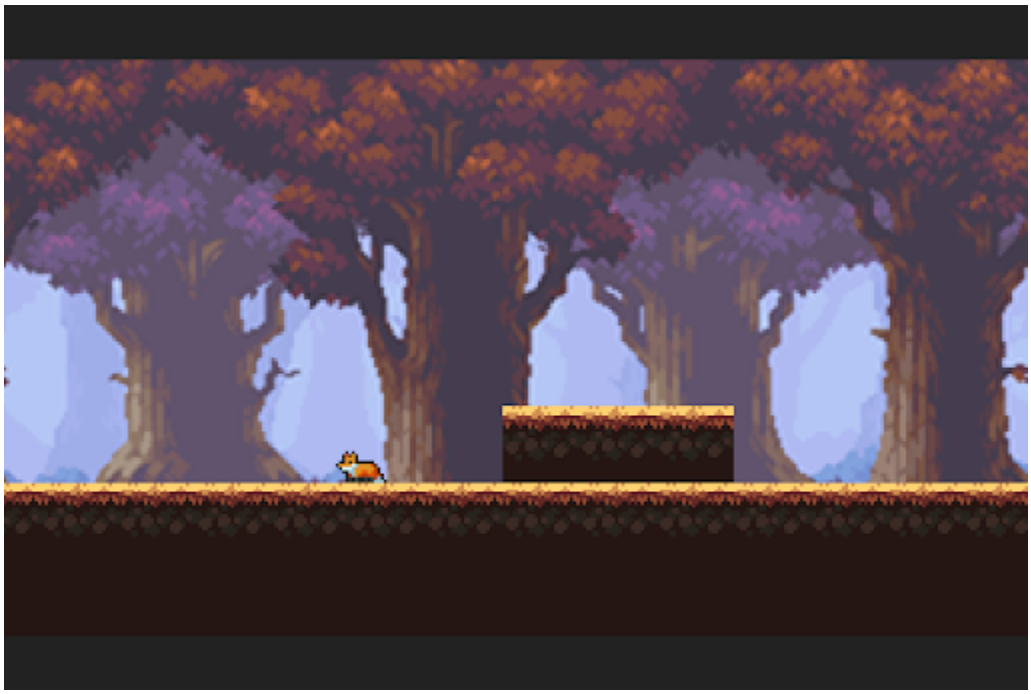
```
  fill(255);
  textSize(10);
  textAlign(LEFT, TOP);
  text("DEBUG MENU (H to hide)", 15, 15);
  text("1. Moon Gravity: " + (moonGravity ? "ON" : "OFF"), 15, 30);
  text("2. Show Hitboxes: " + (showHitboxes ? "ON" : "OFF"), 15, 45);
  text("Player Y-Vel: " + player.vel.y.toFixed(2), 15, 60);
  camera.on();
}
```

Summary of Features Added:

- **Moon Gravity:** Toggles `world.gravity.y` between a low value (2) and the default `GRAVITY` constant.
- **Show Hitboxes:** Uses the p5play `allSprites.debug` property to make collision boxes visible, helping you verify transitions and collisions as suggested in the slides.
- **Real-time State Inspection:** Displays the player's current vertical velocity to help with jump tuning.

What I Gave AI:

I am not seeing anything different on the screen, the keypressed for 1 and 2 don't do anything either. What might be the issue?



What AI Gave Me:

Looking at your screenshot, I see the issue! It looks like you've added the **logic** for the debug menu, but there are two main reasons why it's not appearing or responding to your key presses:

1. The Variable Scope (The "Where")

In your code, you defined `let showDebug = false;` **inside** the `draw()` function.

Why this is a problem: The `draw()` function runs 60 times per second. Every single time it runs, it resets `showDebug` back to `false`. When you press 'H', it might flicker to `true` for a microsecond, but then the very next frame resets it.

The Fix: Move these variables to the very **top** of your script (outside of any function):

JavaScript

```
let showDebug = false;

let moonGravity = false;

let showHitboxes = false;
```

---

2. The "Layering" Issue (The `camera.off()`)

In p5play, the camera follows the game world. If you draw text while the camera is "on," the text stays at a specific coordinate in the level (like `x: 10, y: 10`). If your player moves away, the text stays behind!

The Fix: Ensure your UI code uses `camera.off()` so it "sticks" to your screen like a sticker on a window.

---

3. Your `keyPressed` Logic

Make sure your `keyPressed()` function looks exactly like this. Note that `key` is case-sensitive, so it's safer to check for both 'h' and 'H'.

JavaScript

```
function keyPressed() {
  startMusicIfNeeded(); // Keep your existing music trigger


  // Toggle the menu visibility
  if (key === 'h' || key === 'H') {
    showDebug = !showDebug;
  }


  // Only allow 1 and 2 to work if the menu is actually open
  if (showDebug) {
    if (key === '1') {
      moonGravity = !moonGravity;
```

```
    }
    if (key === '2') {
      showHitboxes = !showHitboxes;
    }
  }
}
```

---

4. Updating Gravity in Real-Time

Inside your `draw()` function, you need to actually **apply** the moon gravity logic, otherwise, the variable changes but the physics engine doesn't know.

JavaScript

```
function draw() {
  // Add this at the start of draw()
  if (moonGravity) {
    world.gravity.y = 2; // Low gravity
  } else {
    world.gravity.y = 10; // Normal gravity
  }

  allSprites.debug = showHitboxes;

  // ... rest of your code ...
}
```

Quick Troubleshooting Tip:

Open your browser's **Console** (F12 or Right Click > Inspect > Console). If you see red text, you might have a typo in a variable name (like `showdebug` vs `showDebug`) which will stop the code from running entirely.

Would you like me to provide the full, corrected `sketch.js` file so you can copy-paste it and see it in action?

What I Gave AI:

Nothing has changed even with these updates above, however when inspecting browaser console, it is showing me "Failed to load resource: the server responded with a status of 404 (Not Found)Understand this error" What might be the issue here? Here is my current code:
index.html <!doctype html>

```
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <title>GBDA302 Week 9 - Example 3: Adding Sound & Music</title>
    <style>
      body {
        margin: 0;
        overflow: hidden;
        background: #222;
      }
      canvas {
        display: block;
      }
    </style>
  </head>
  <body>
    <script src="https://cdn.jsdelivr.net/npm/p5@1.11.4/lib/p5.min.js"></script>
    <script src="https://cdn.jsdelivr.net/npm/p5@1.11.4/lib/addons/p5.sound.min.js"></script>
    <script src="https://p5play.org/v3/planck.min.js"></script>
    <script src="https://p5play.org/v3/p5play.min.js"></script>
    <script src="sketch.js"></script>
  </body>
</html>
```

jsconfig.json

```
{
  "compilerOptions": {
```

```
    "target": "es6",
    "checkJs": false,
    "allowJs": true,
  },
  "include": [
    "*.js",
    "**/*.js",
    "./types/p5/index.d.ts",
    "./types/p5/global.d.ts"
  ],
   "exclude": ["node_modules"]
}
sketch.js /*
  Week 9 — Example 3: Adding Sound & Music

  Course: GBDA302 | Instructors: Dr. Karen Cochrane & David Han
  Date: Mar. 19, 2026

  Controls:
    A or D (Left / Right Arrow)   Horizontal movement
    W (Up Arrow)                  Jump
    Space Bar                     Attack

  Tile key:
    g = groundTile.png       (surface ground)
    d = groundTileDeep.png   (deep ground, below surface)
      = empty (no sprite)
*/

let player;
let playerImg, bgImg;
let jumpSfx, musicSfx;
let musicStarted = false;

let showDebug = false; // Toggle the menu visibility
let moonGravity = false; // Toggle for moon gravity
let showHitboxes = false; // Additional feature: view collision boxes

let playerAnis = {
  idle: { row: 0, frames: 4, frameDelay: 10 },
  run: { row: 1, frames: 4, frameDelay: 3 },
  jump: { row: 2, frames: 3, frameDelay: Infinity, frame: 0 },
  attack: { row: 3, frames: 6, frameDelay: 2 },
};
```

```
let ground, groundDeep;
let groundImg, groundDeepImg;

let attacking = false; // track if the player is attacking
let attackFrameCounter = 0; // tracking attack animation

// --- TILE MAP ---
// an array that uses the tile key to create the level
let level = [
  "              ",
  "              ",
  "              ",
  "              ",
  "              ",
  "       ggg    ",
  "gggggggggggggg", // surface ground
  "dddddddddddddd", // deep ground
];

// --- LEVEL CONSTANTS ---
// camera view size
const VIEWW = 320,
  VIEWH = 180;

// tile width & height
const TILE_W = 24,
  TILE_H = 24;

// size of individual animation frames
const FRAME_W = 32,
  FRAME_H = 32;

// Y-coordinate of player start (4 tiles above the bottom)
const MAP_START_Y = VIEWH - TILE_H * 4;

// gravity
const GRAVITY = 10;

function preload() {
  // --- IMAGES ---
  playerImg = loadImage("assets/foxSpriteSheet.png");
  bgImg = loadImage("assets/combinedBackground.png");
  groundImg = loadImage("assets/groundTile.png");
```

```
  groundDeepImg = loadImage("assets/groundTileDeep.png");

  // --- SOUND ---
  if (typeof loadSound === "function") {
    jumpSfx = loadSound("assets/sfx/jump.wav");
    musicSfx = loadSound("assets/sfx/music.wav");
  }
}

function setup() {
  // pixelated rendering with autoscaling
  new Canvas(VIEWW, VIEWH, "pixelated");

  // needed to correct an visual artifacts from attempted antialiasing
  allSprites.pixelPerfect = true;

  world.gravity.y = GRAVITY;

  // Try to start background music immediately.
  if (musicSfx) musicSfx.setLoop(true);
  startMusicIfNeeded();

  // --- TILE GROUPS ---
  ground = new Group();
  ground.physics = "static";
  ground.img = groundImg;
  ground.tile = "g";

  groundDeep = new Group();
  groundDeep.physics = "static";
  groundDeep.img = groundDeepImg;
  groundDeep.tile = "d";

  // a Tiles object creates a level based on the level map array (defined at the beginning)
  new Tiles(level, 0, 0, TILE_W, TILE_H);

  // --- PLAYER ---
  player = new Sprite(FRAME_W, MAP_START_Y, FRAME_W, FRAME_H); // create the player
  player.spriteSheet = playerImg; // use the sprite sheet
  player.rotationLock = true; // turn off rotations (player shouldn't rotate)

  // player animation parameters
  player.anis.w = FRAME_W;
  player.anis.h = FRAME_H;
```

```
  player.anis.offset.y = -4; // offset the collision box up
  player.addAnis(playerAnis); // add the player animations defined earlier
  player.ani = "idle"; // default to the idle animation
  player.w = 18; // set the width of the collsion box
  player.h = 20; // set the height of the collsion box
  player.friction = 0; // set the friciton to 0 so we don't stick to walls
  player.bounciness = 0; // set the bounciness to 0 so the player doesn't bounce

  // --- GROUND SENSOR --- for use when detecting if the player is standing on the ground
  sensor = new Sprite();
  sensor.x = player.x;
  sensor.y = player.y + player.h / 2;
  sensor.w = player.w;
  sensor.h = 2;
  sensor.mass = 0.01;
  sensor.removeColliders();
  sensor.visible = false;
  let sensorJoint = new GlueJoint(player, sensor);
  sensorJoint.visible = false;
}

function startMusicIfNeeded() {
  if (musicStarted || !musicSfx) return;

  const startLoop = () => {
    if (!musicSfx.isPlaying()) musicSfx.play();
    musicStarted = musicSfx.isPlaying();
  };

  // Some browsers require a user gesture before audio can start.
  const maybePromise = userStartAudio();
  if (maybePromise && typeof maybePromise.then === "function") {
    maybePromise.then(startLoop).catch(() => {});
  } else {
    startLoop();
  }
}

function keyPressed() {
  startMusicIfNeeded();

  if (key === "h" || key === "H") {
    showDebug = !showDebug; // Toggle debug menu visibility
  }
```

```
  if (showDebug) {
    if (key === "1") moonGravity = !moonGravity; // Toggle Moon Gravity
    if (key === "2") showHitboxes = !showHitboxes; // Toggle Hitboxes
  }
}

function mousePressed() {
  startMusicIfNeeded();
}

function touchStarted() {
  startMusicIfNeeded();
  return false;
}

function draw() {
  if (moonGravity) {
    world.gravity.y = 2;
  } else {
    world.gravity.y = 10;

    allSprites.debug = showHitboxes;
  }
  // --- BACKGROUND ---
  camera.off();
  imageMode(CORNER);
  image(bgImg, 0, 0, bgImg.width, bgImg.height);
  camera.on();

  // --- PLAYER CONTROLS ---
  // first check to see if the player is on the ground
  let grounded = sensor.overlapping(ground);

  // -- ATTACK INPUT --
  if (grounded && !attacking && kb.presses("space")) {
    attacking = true;
    attackFrameCounter = 0;
    player.vel.x = 0;
    player.ani.frame = 0;
    player.ani = "attack";
    player.ani.play(); // plays once to end
  }
  // Toggle gravity based on debug setting
```

```
if (moonGravity) {
  world.gravity.y = 2; // Low gravity (Moon)
} else {
  world.gravity.y = GRAVITY; // Normal gravity (Earth)
}

// Toggle hitbox visibility for all sprites
allSprites.debug = showHitboxes;
// -- JUMP --
if (grounded && kb.presses("up")) {
  player.vel.y = -4;
  if (jumpSfx) jumpSfx.play();
}

// --- STATE MACHINE ---
if (attacking) {
  attackFrameCounter++;
  // Attack lasts ~6 frames * frameDelay 2 = 12 cycles (adjust if needed)
  if (attackFrameCounter > 12) {
    attacking = false;
    attackFrameCounter = 0;
  }
} else if (!grounded) {
  player.ani = "jump";
  player.ani.frame = player.vel.y < 0 ? 0 : 1;
} else {
  player.ani = kb.pressing("left") || kb.pressing("right") ? "run" : "idle";
}

// --- MOVEMENT ---
if (!attacking) {
  player.vel.x = 0;
  if (kb.pressing("left")) {
    player.vel.x = -1.5;
    player.mirror.x = true;
  } else if (kb.pressing("right")) {
    player.vel.x = 1.5;
    player.mirror.x = false;
  }
}

// --- KEEP IN VIEW ---
player.pos.x = constrain(player.pos.x, FRAME_W / 2, VIEWW - FRAME_W / 2);
```

```
  // --- DEBUG MENU UI ---
  if (showDebug) {
    camera.off(); // Ensure the menu stays fixed on screen
    fill(0, 0, 0, 150); // Semi-transparent background
    rect(10, 10, 150, 70, 5);

    fill(255);
    textSize(10);
    textAlign(LEFT, TOP);
    text("DEBUG MENU (H to hide)", 15, 15);
    text("1. Moon Gravity: " + (moonGravity ? "ON" : "OFF"), 15, 30);
    text("2. Show Hitboxes: " + (showHitboxes ? "ON" : "OFF"), 15, 45);
    text("Player Y-Vel: " + player.vel.y.toFixed(2), 15, 60);
    camera.on();
  }
}
style.css html, body {
  margin: 0;
  padding: 0;
}

canvas {
  display: block;
}
```

What AI Gave Me:

The **404 (Not Found)** error in your console is the main reason you aren't seeing any changes. In p5.js, if the program fails to load a single asset (like a sound or an image), it often "hangs" or stops executing the rest of the script to prevent further errors.

Looking at your code, the 404 is likely happening in the `preload()` function because the file paths for your sounds or images don't match your folder structure.

How to fix the 404 and the Debug Menu:

1. Fix the "Hanging" Script (Preload)

If you don't actually have the sound files yet, **comment them out**. p5.js will stop running if it can't find `assets/sfx/jump.wav`.

JavaScript
```
function preload() {
```

```
  playerImg = loadImage("assets/foxSpriteSheet.png");
  bgImg = loadImage("assets/combinedBackground.png");
  groundImg = loadImage("assets/groundTile.png");
  groundDeepImg = loadImage("assets/groundTileDeep.png");

  // COMMENT THESE OUT if the files are missing from your folder
  // jumpSfx = loadSound("assets/sfx/jump.wav");
  // musicSfx = loadSound("assets/sfx/music.wav");
}
```

2. Fix the UI Drawing Order

In your current `draw()` function, you have `camera.on()` right before your debug menu code. Because the player is likely at the bottom of the screen (Y = 150+), drawing text at `15, 15` while the camera is **ON** means the text is being drawn way off-screen at the very top of the game world.

Change your UI section to this (ensure `camera.off()` is called correctly):

JavaScript
```
 // --- DEBUG MENU UI ---
  if (showDebug) {
    camera.off(); // Turn off camera so we draw on the SCREEN, not the WORLD

    fill(0, 0, 0, 150);
    rect(10, 10, 150, 70, 5);

    fill(255);
    noStroke(); // Prevents weird outlines on text
    textSize(10);
    textAlign(LEFT, TOP);
    text("DEBUG MENU (H to hide)", 15, 15);
    text("1. Moon Gravity: " + (moonGravity ? "ON" : "OFF"), 15, 30);
    text("2. Show Hitboxes: " + (showHitboxes ? "ON" : "OFF"), 15, 45);
    text("Player Y-Vel: " + player.vel.y.toFixed(2), 15, 60);

    // Do NOT call camera.on() here if it's the last thing in draw()
  }
```

3. Clean up the Duplicate Logic

You have the gravity toggle code written twice inside your `draw()` function. While it won't break the game, it makes it harder to read. Delete the first instance at the very top of `draw()` and keep the one near the movement logic.

Why you don't see anything yet:

1. **The 404 Crash:** The browser stops the script before it even gets to `setup()`. Fix the file paths or comment out the `loadSound` lines.
2. **Focus:** Click on the game canvas with your mouse before pressing **'H'**. If the browser window isn't "focused" on the canvas, it won't register your key presses.

Try this sequence:

1. Comment out the `loadSound` lines in `preload`.
2. Refresh the page.
3. Check the console—if it's empty (no red text), the game is running.
4. **Click the game screen.**
5. Press **'H'**.